Biomedical Signal Processing and Control 113 (2026) 109251

Contents lists available at ScienceDirect

Biomedical Signal Processing and Control

journal homepage: www.elsevier.com/locate/bspc

MedPrompt: LLM–CNN Fusion with Weight Routing for Medical Image Segmentation and Classification

Shadman Sobhan [a], Kazi Abrar Mahmud [a], Abduz Zami [b], Mohiuddin Ahmed [b], Md Palash Uddin [c,d,*]

[a] Department of Electrical and Electronic Engineering, Bangladesh University of Engineering and Technology, Dhaka 1000, Bangladesh
[b] Department of Computer Science & Engineering, Rajshahi University of Engineering & Technology, Rajshahi 6204, Bangladesh
[c] School of Information Technology, Deakin University, Geelong, VIC 3220, Australia
[d] Department of Computer Science and Engineering, Hajee Mohammad Danesh Science and Technology University, Dinajpur, 5200, Bangladesh

ARTICLE INFO

Keywords:
Medical image analysis
Few-shot prompting
LLM–CNN fusion
Weight routing
Segmentation
Classification

ABSTRACT

Most existing medical image analysis systems are task-specific, requiring separate models for classification and segmentation, and lack the flexibility to support user-defined workflows expressed in natural language. Developing a prompt-driven medical assistant introduces significant challenges, including the need for large annotated datasets, integration of multimodal data (text and images), high computational demands, and the difficulty of adapting to diverse tasks without retraining. To address these challenges, we present MedPrompt, a unified and extensible framework that combines a few-shot prompted Large Language Model (LlaMA-4-17B) for task planning with a modular CNN architecture (DeepFusionLab) for image analysis. The LLM parses user instructions and generates structured JSON workflows, dynamically selecting and routing task-specific pretrained weights to DeepFusionLab—a strategy we term *dynamic weight routing*. This eliminates the need for retraining when adding new tasks and supports multi-stage and conditional task execution. We evaluate MedPrompt on 19 public datasets across 12 tasks and 5 imaging modalities. The system achieves 97% end-to-end correctness in interpreting and executing natural language prompts, with an average inference latency of 2.5 s on CPU—suitable for near real-time clinical applications. DeepFusionLab demonstrates strong segmentation accuracy (e.g., Dice ∼0.9856 for lungs) and classification performance (F1 ∼0.9744 for tuberculosis). Overall, MedPrompt offers a scalable, interpretable, and deployment-ready solution for prompt-driven medical image analysis across a wide range of clinical scenarios.

1. Introduction

Natural language is the most common mode of human communication. In contrast, medical diagnostics are primarily based on imaging modalities, including X-rays, CT scans, MRI, and endoscopy [1,2]. Convolutional Neural Networks (CNNs), designed for image-based tasks, have become a cornerstone of medical image analysis [3–5]. The core tasks in this domain include *segmentation* — localizing organs or lesions — and *classification* — identifying disease presence [6–8]. While multitask vision models have emerged, few support segmentation via natural language prompts. Notable examples like the Segment Anything Model (SAM) [9] and MedSAM [10] enable zero-shot segmentation using spatial prompts (points or bounding boxes), but suffer from three core limitations: (i) spatial prompts are less intuitive and harder to automate than textual prompts, (ii) fine-tuning for new anatomy demands extensive data and compute, and (iii) these models focus solely on segmentation and require additional training for classification tasks.

Recent advances in Large Language Models (LLMs) have enabled machines to understand and reason over text with human-like proficiency [11,12]. Similarly, Vision–Language Models (VLMs) now show strong multimodal reasoning capabilities [13]. However, most VLMs lack deep integration with image processing backbones, which limits their application in dense prediction tasks, such as segmentation. General-purpose medical models aim to support a wide range of tasks, yet this often comes at the cost of performance. In clinical practice, task-specific accuracy is prioritized over generalization. Nguyen et al. [14] showed that models like SAM, even after fine-tuning, underperform specialized models like nnU-Net by 10%–15% on certain tasks.

* Corresponding author at: School of Information Technology, Deakin University, Geelong, VIC 3220, Australia.
E-mail addresses: shadmansobhan114@gmail.com (S. Sobhan), abrar.mahmud790011@gmail.com (K.A. Mahmud), abduz.zami@gmail.com (A. Zami), mohiuddin.nirob.mn@gmail.com (M. Ahmed), m.uddin@deakin.edu.au (M.P. Uddin).

https://doi.org/10.1016/j.bspc.2025.109251
Received 6 August 2025; Received in revised form 20 October 2025; Accepted 16 November 2025
Available online 20 November 2025
1746-8094/© 2025 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

Moreover, generalized models such as SAM (91M–636M parameters) are far more resource-intensive than task-specific CNNs.

To overcome these limitations, we introduce MedPrompt—a prompt-driven framework combining a few-shot LLM and a modular CNN (DeepFusionLab). The LLM parses user instructions, interprets the target task, and dynamically selects the appropriate pretrained CNN weights for execution. This strategy, termed *weight routing*, ensures high performance on specific medical tasks while preserving flexibility and scalability. Adding a new task only requires training DeepFusionLab and registering its weights; no system-wide changes are necessary. Our key contributions are as follows:

- We propose MedPrompt, a unified framework that integrates a few-shot prompted LLM with a modular CNN architecture (DeepFusionLab) to perform both segmentation and classification from natural language prompts. This enables prompt-driven, user-friendly medical image analysis without the need to configure or retrain manual tasks.
- We introduce a novel dynamic weight routing mechanism that interprets intent, target, and modality of the commands to load pretrained task-specific weights on the fly. This allows scalable task execution, supports complex multistage and conditional workflows, and eliminates the need for full model retraining when adding new tasks.
- We design and implement DeepFusionLab, a lightweight, dual-mode CNN capable of efficient classification and segmentation using a shared encoder and task-specific branches. Extensive experiments in public datasets, 12 tasks, and 5 imaging modalities show high precision and generalizability, with near real-time latency (2.5s on CPU), demonstrating the suitability of MedPrompt for clinical deployment.

The rest of the paper is organized as follows: Section 2 reviews related works and outlines key research gaps. Section 3 details the MedPrompt framework, including the DeepFusionLab architecture and dynamic weight routing. Section 4 describes the experimental setup, and Section 5 presents evaluation results and ablation studies. Section 6 concludes with key insights and future directions.

2. Literature review

CNNs have been widely adopted for medical image classification and segmentation. Architectures such as ResNet [15], DenseNet [16], EfficientNet [17], and Vision Transformers (ViT) [18] have significantly advanced feature representation and computational efficiency. These models have been adapted to diagnose conditions like COVID-19 [19], diabetic retinopathy [20], and CheXNet [21] for pneumonia. For segmentation tasks, encoder–decoder models such as FCN [22] and SegNet [23] introduced pixel-wise prediction capabilities. U-Net [24] became the de facto standard in biomedical segmentation due to its skip connections, while DeepLabv3+ [25] improved multi-scale context capture. Hybrid architectures such as TransUNet [26] and Swin-Unet [27] combined transformers with CNNs to enhance global context understanding, achieving state-of-the-art results on tasks like tumor and vessel segmentation [28,29].

While CNNs are traditionally optimized for either segmentation or classification, some multi-task models integrate both. Mask R-CNN [30] performs simultaneous detection and segmentation. More recent multi-task systems such as Swin UNETR [31] and COVID-CT-Mask-Net [32] incorporate transformer or CNN backbones to jointly address classification and segmentation tasks.

Promptable segmentation models like SAM [9] and its medical adaptation MedSAM [10] allow user interaction via spatial prompts. However, they are primarily segmentation-focused, and adapting them for classification tasks requires further fine-tuning. Advanced models such as SEEM [33], LISA [34], and GLaMM [35] employ multimodal prompts and attention mechanisms for open-world segmentation, but often lack support for detailed medical workflows.

VLMs offer zero-shot classification and retrieval by aligning text with images. Models like MedCLIP [36] and BioViL [37] use contrastive learning and domain-specific pretraining to improve multimodal understanding. Hybrids such as MedCLIP-SAM [38] and LLaVA-Med [39] bridge LLMs with visual backbones, enabling prompt-based interaction. Large models like BiomedGPT [40] and GPT-4V are also being explored for generalized clinical tasks. Instruction-driven systems like VisionLLM v2 [41] connect LLMs with task-specific decoders to unify VQA, localization, and editing. SAM4MLLM [42] integrates SAM with LLMs for referring segmentation. Zeus [43] leverages a frozen LLM to generate segmentation instructions, while InstructSeg [44] supports both reasoning-based and referring segmentation via text prompts.

Despite significant advancements in medical image analysis, several critical limitations persist. Most CNN-based models are task-specific, requiring separate architectures for classification and segmentation, which complicates clinical workflow integration. Prompt-based models like SAM rely on spatial inputs that are less intuitive than natural language, reducing usability in clinical contexts. Meanwhile, VLMs such as MedCLIP support classification but lack dense segmentation capabilities. Additionally, current systems often lack scalability; adapting to new tasks typically necessitates costly retraining. Finally, many models suffer from high inference latency, limiting their applicability in real-time clinical scenarios. To address these challenges, we propose MedPrompt—a scalable, extensible framework that combines a few-shot LLM with a modular CNN. MedPrompt supports both classification and segmentation via natural language instructions, enables dynamic weight routing and conditional logic, and achieves clinically acceptable latency for real-time deployment.

3. Methodology

The methodology begins with the LLM receiving a user-defined prompt and extracting key information such as the intended task (e.g., classification or segmentation), the target organ, and the specific disease or condition. The LLM has access to a repository of pretrained weights, each associated with a specific task, which were generated by training the CNN backbone, DeepFusionLab. Leveraging this knowledge, the LLM constructs a structured routine in JSON format that iteratively invokes DeepFusionLab in the appropriate mode (segmentation or classification) and loads the corresponding task-specific weights. This process enables dynamic, prompt-driven task execution. An overview of the proposed framework architecture is illustrated in Fig. 1. For clarity and ease of explanation, the methodology is organized into three main components: (i) Framework Inputs, (ii) Language Model-Based Text Processing and Structured Output Generation, and (iii) Coordination of Structured Input with DeepFusionLab.

3.1. Framework inputs

The proposed MedPrompt framework requires three essential inputs: a natural language prompt, a medical image, and access to a weight database. The prompt is a user-defined natural language instruction that specifies the desired operation and the target organ or disease. Examples of such prompts include: “Isolate the lung region”, “Do you find pneumonia in the given image?” or “This is a person’s report. Does the person have malaria?” The weight database contains pretrained model weights generated by training DeepFusionLab on various classification and segmentation tasks. To ensure accurate interpretation by the LLM, these weights are expected to follow a standardized naming convention. The recommended naming format is illustrated in Table 1. Strict adherence to the naming convention is not mandatory. Variants such as “Cls”, “Class”, or “Classification” are acceptable, as long as the overall format `intent_target_modality` remains clear and semantically consistent. The third input is the medical image on which the specified segmentation or classification task will be performed.

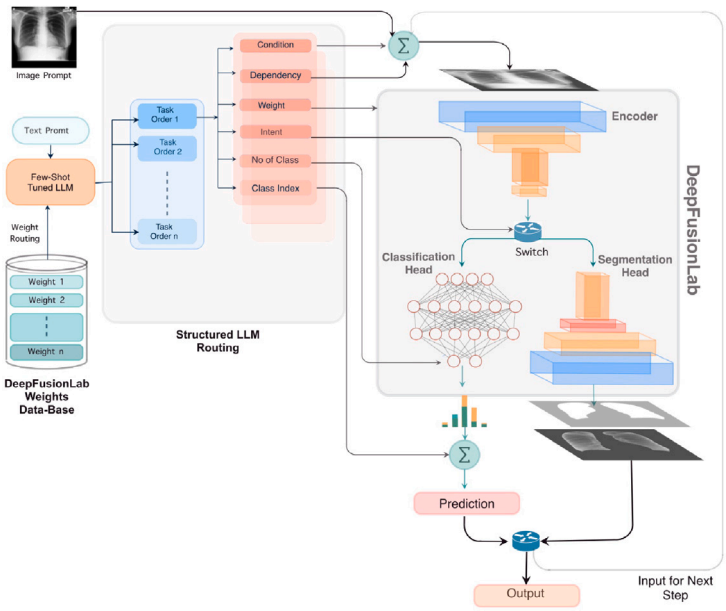


Fig. 1. MedPrompt architecture.

Table 1
Standardized weight naming format for segmentation and classification tasks.

Type of weight	Standard format	Example
Segmentation	Seg_Organ/Disease_Modality/ Segmentation_Organ_Modality	Seg_Lung_Chest X-ray Segmentation_Lung_CXR
Binary Classification	Cls_Disease_Modality/ Classification_Disease_Modality	Cls_TB_Chest X-ray Classification_Tuberculosis_CXR
Multi-class Classification	Cls_Disease1-Disease2-..._Modality	Cls_Covid-Pneumonia_CXR

3.2. Prompt parsing and structured workflow generation

Fig. 2 illustrates the methodology for structured output generation by the LLM. The process begins by retrieving the names of all available pretrained model weights from a centralized weight database. Each filename encodes three key attributes: the task intent (*classification* or *segmentation*), the target organ or condition, and the imaging modality. These metadata components are extracted and used to compile reference lists of valid targets and modalities. A few-shot prompted LLM — specifically, LlaMA-4-17B — interprets the user's natural language prompt and decomposes it into a sequence of structured tasks. Each task is defined by an *intent*, *target*, and, optionally, a *modality*. These fields are then normalized against the reference lists derived from the weight database. Normalization is performed in two stages. First, a rule-based dictionary resolves common term variations and synonyms (e.g., "tuberculosis" → "tb"). For terms not covered by the dictionary, the system defaults to LLM-based semantic normalization. For instance, a prompt containing "phthisis" is semantically matched to "tuberculosis." This hybrid strategy ensures robustness and flexibility in handling diverse clinical language. A similar normalization approach is applied to the modality field when specified.

The framework also supports conditional execution. If the user prompt implies that a task should be performed only under specific conditions — such as following a positive or negative result from a prior task — the system encodes these dependencies within the structured workflow. Once all tasks are parsed and normalized, the framework performs *weight routing*, which selects the most appropriate pretrained model for each task. This selection is based on a computed *match score* for each candidate weight, considering three criteria: intent match, target similarity, and modality similarity. Semantic similarities are calculated using cosine similarity between Sentence-BERT embeddings of the task descriptors and the corresponding weight metadata. The final score S for a given weight is computed as:

$$S = I + \alpha \cdot \text{sim}_{\text{target}} + \beta \cdot \text{sim}_{\text{modality}}, \tag{1}$$

where $I = 1$ if the task intent matches the weight intent, and $I = 0$ otherwise; $\text{sim}_{\text{target}} \in [0, 1]$ represents the cosine similarity between the task and weight targets; $\text{sim}_{\text{modality}} \in [0, 1]$ denotes the similarity between the modalities; and the weighting factors are set as $\alpha = 1.5$ and $\beta = 1.0$. We set the target similarity weight $\alpha = 1.5$ to prioritize target matching over modality when selecting pretrained weights. This value was determined empirically by evaluating different values in the range $[1.2, 2.0]$ on a validation set of user prompts. A value of $\alpha = 1.5$ consistently produced the most accurate and reliable matches. The maximum possible match score is:

$$S_{\max} = 1 + 1.5 \cdot 1 + 1.0 \cdot 1 = 3.5. \tag{2}$$

A weight is considered a valid match only if its score exceeds a predefined threshold of 1.6, i.e., $S > 1.6$. This threshold was determined by analyzing score distributions and selection outcomes across a set of annotated prompt–task pairs. A value of 1.6 reliably filtered out weak or spurious matches while retaining all correct ones. Given that the maximum possible score is 3.5, this threshold allows for partial matches — such as correct targets with uncertain modalities — while avoiding unrelated or misleading weights. Lower thresholds were found to increase the rate of false positives, particularly in cases where only the modality matched semantically. When multiple classification weights exist for the same target, the system naturally prefers binary classifiers over multiclass models. This preference arises from higher semantic similarity scores, as binary classifiers are typically more focused and demonstrate superior diagnostic performance. For classification tasks, the number of classes and their corresponding labels are inferred

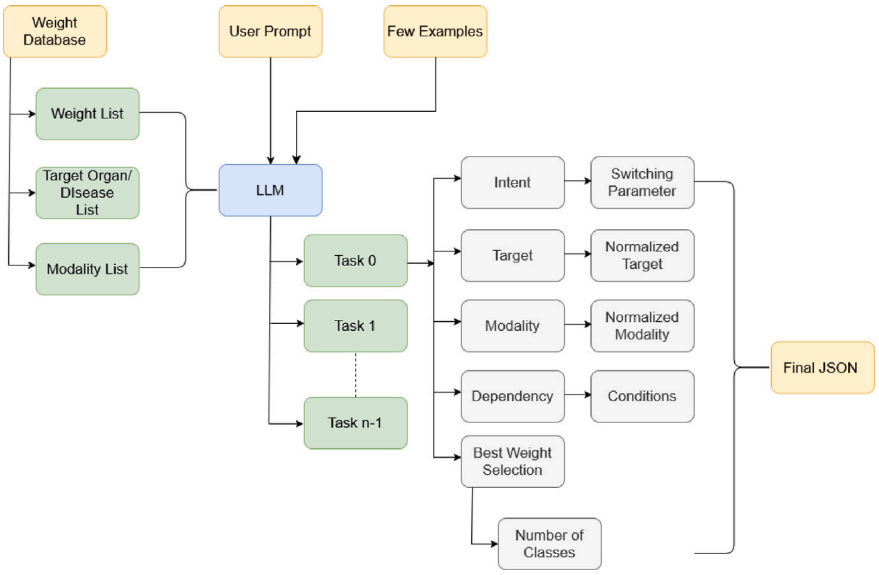


Fig. 2. Text processing and structured output generation.

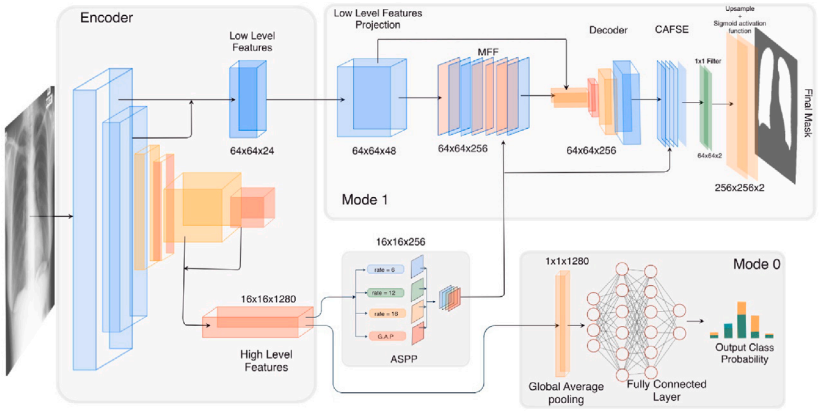


Fig. 3. DeepFusionLab architecture.

from either the filename or an associated JSON mapping file. For segmentation tasks, a default two-class setup is assumed, representing the background and the target region. The final structured JSON output includes each task's resolved intent, target, modality, dependencies, conditional logic (if applicable), selected weight, and class-level information.

3.3. DeepFusionLab architecture

The proposed methodology employs DeepFusionLab, a CNN architecture designed to support both classification and segmentation tasks through a shared encoder and task-specific decoding branches. As illustrated in Fig. 3, the model accepts input images of size $256 \times 256 \times 3$ and adapts its behavior based on a binary mode flag: `Mode = 0` for classification and `Mode = 1` for segmentation.

3.3.1. Encoder (feature extractor)

The encoder in DeepFusionLab uses EfficientNet-B0, pretrained on ImageNet, as its backbone. EfficientNet-B0 was selected for its optimal balance between computational efficiency and accuracy. Two levels of features are extracted. (i) Low-Level Features: Extracted from the early layers of the network, these feature maps have a spatial resolution of 64×64 with 24 channels. They capture fine-grained details and are particularly useful for segmentation tasks. (ii) High-Level Features: Obtained from deeper layers, these feature maps have a resolution of 16×16 with 1280 channels. They encode more abstract, semantic information and are useful for both classification and segmentation.

3.3.2. Classification pathway

When the mode is set to 0, the model performs classification. Global average pooling is applied on the high-level features, and the resulting feature map is flattened into a single vector that summarizes the entire image. This vector is passed through a dense layer with ReLU activation and dropout for regularization, followed by a final dense layer that produces class scores. A softmax activation function converts these scores into class probabilities.

3.3.3. Segmentation pathway

In segmentation mode (`Mode = 1`), both low-level and high-level features are utilized. The low-level features are processed through a 1×1 convolution to increase the number of channels from 24 to 48, resulting in a richer feature representation known as low-level feature projections. In parallel, the high-level features are passed through the ASPP module. These two branches are then fused using the Multi-Feature Fusion (MFF) block. The resulting fused feature map, along with the low-level projections, is passed through a decoder to progressively recover spatial resolution. To further refine the output, a Coarse and Fine Step Enhancement (CAFSE) block is used to combine decoder

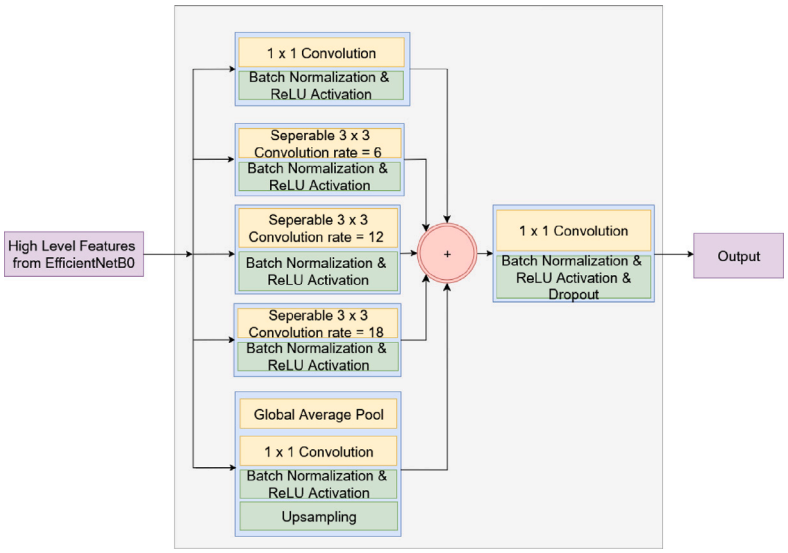


Fig. 4. ASPP block architecture.

output with ASPP features using channel-wise attention. Finally, a 1×1 convolution reduces the output to two channels corresponding to the background and the target region. Bilinear interpolation upsamples the output to match the input image resolution, and a sigmoid activation function produces the final probability mask, indicating the likelihood of each pixel belonging to the target region.

3.3.4. ASPP block architecture

The ASPP block is designed to extract rich, multi-scale contextual features by applying convolutional operations with various dilation rates. As depicted in Fig. 4, the ASPP module includes parallel branches: a 1×1 convolution, multiple 3×3 depthwise separable convolutions with different atrous (dilation) rates, and a global average pooling branch followed by a 1×1 convolution. These outputs are concatenated and passed through a 1×1 projection convolution to unify the channel dimensions, followed by a dropout layer to prevent overfitting. The ASPP block effectively captures both local and global context.

3.3.5. MFF block architecture

The MFF block, shown in Fig. 5, is designed to effectively combine high-level and low-level features extracted from different stages of the encoder. High-level features, typically from the ASPP output, are first upsampled to match the spatial resolution of the low-level features. Both feature maps are then passed through 1×1 convolutional layers for channel projection and subsequently summed. The combined output is further refined using a 3×3 convolution followed by batch normalization and ReLU activation. To enhance representational power, a Squeeze-and-Excitation (SE) block [45] is applied, providing channel-wise attention. This mechanism allows MFF to adaptively emphasize informative channels while suppressing less relevant ones, resulting in improved feature integration for downstream decoding.

3.3.6. Decoder architecture

The decoder, illustrated in Fig. 6, is responsible for progressively recovering the spatial details that are lost during the encoding process. It begins by upsampling the input feature map from the MFF block to match the spatial resolution of the low-level features extracted earlier in the encoder. A 1×1 convolution is then applied to the upsampled features, followed by batch normalization and ReLU activation. These processed features are concatenated with the corresponding low-level features from the encoder, facilitating the integration of semantic and spatial information. The concatenated feature map is further refined through two depthwise separable convolution layers, each interleaved with batch normalization, ReLU activation, and variable dropout. This design enables the decoder to reconstruct high-resolution outputs while maintaining computational efficiency.

3.3.7. CAFSE architecture

The CAFSE block, depicted in Fig. 7, is designed to fuse high-level semantic features from the ASPP block (coarse) with spatially detailed features from the decoder (fine). The coarse features provide rich contextual information but are low in resolution, while the fine features offer detailed spatial information but lack global context. To align their spatial dimensions, the ASPP output is first upsampled to match the resolution of the decoder output. It is then passed through a 1×1 convolution followed by a depthwise separable 3×3 convolution to enhance its semantic representation. In parallel, the decoder output undergoes a 1×1 convolution followed by a sigmoid activation to generate an attention map. This attention map modulates the enhanced coarse features through element-wise multiplication, emphasizing relevant regions. The result is then added to the original decoder features, producing a refined feature map that integrates both semantic and spatial cues. CAFSE effectively balances semantic abstraction with spatial precision, a crucial capability in medical image segmentation. In DeepFusionLab, separable convolutions are used in place of standard convolutions to reduce computational complexity, ensuring the model remains lightweight yet effective. Table 2 summarizes the parameter count and operational roles of each component in the proposed DeepFusionLab architecture.

4. Experimental setup

This section details the experimental framework used to prepare, train, and evaluate our model, ensuring methodological rigor and reproducibility across a diverse range of medical imaging tasks.

4.1. Datasets

The datasets used to train DeepFusionLab are summarized in Table 3. All the datasets and their respective masks or labels are from standard public datasets. All the annotations are from the listed datasets. We prioritized dataset diversity by considering factors such as image count, imaging modality, and task type. This diversity ensures that DeepFusionLab avoids underfitting or overfitting to any specific dataset and enhances MedPrompt's generalizability across a broad range of modalities and clinical tasks.

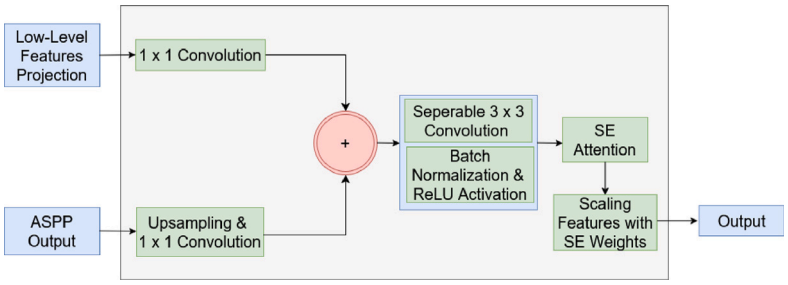


Fig. 5. MFF block architecture.

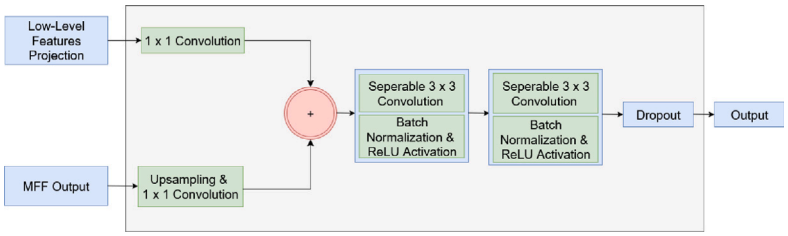


Fig. 6. Decoder architecture.

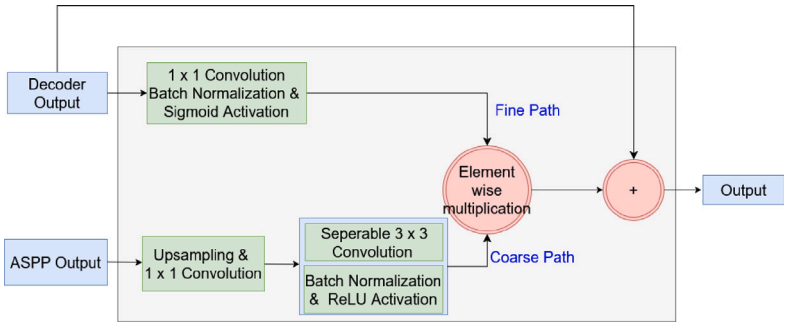


Fig. 7. CAFSE block architecture.

Table 2
Functional roles and parameter count of each major block in the proposed DeepFusionLab architecture.

Block	Parameters	Roles
Backbone (EfficientNet-B0)	4.007 M	Extracts low and high-level features from the input image.
ASPP	2.003 M	Captures multi-scale context using parallel atrous convolutions.
MFF	0.684 M	Fuses low-level and high-level features for spatial refinement.
Decoder	0.106 M	Reconstructs spatial details for mask generation.
CAFSE	0.656 M	Enhances spatial precision through coarse-to-fine feature integration.
Classification Head	0.656 M	Maps global features to final class probabilities.
Segmentation Head	514	Produces pixel-level prediction maps for the target structure.
Overall	**8.12 M**	–

4.2. DeepFusionLab training

Each dataset was initially split into training and testing sets using an 80:20 ratio, ensuring class-balanced splits. The training set was further divided into training and validation subsets in a 90:10 ratio. For segmentation tasks, the model was set to segmentation mode and trained for 200 epochs. Batch size was determined based on the dataset size: a batch size of 8 was used for datasets with fewer than 500 training images, while 16 or 32 was used for larger datasets, depending on memory availability. The Dice Loss function, derived from the Dice Coefficient, was used for optimization. To reduce overfitting, real-time data augmentation was applied during training, including horizontal and vertical flips. For classification tasks, the model was trained for 100 epochs in classification mode. Batch sizes of 32, 64, or 128 were chosen

Table 3
Datasets used in this study.

Purpose	Dataset	Modality	Training/Testing images
Lung Segmentation	JSRT [46]	Chest X-ray	197/50
Lung Segmentation & Tuberculosis Classification	Montgomery [47]		110/28
Tuberculosis Classification	Shenzhen [48]		529/133
	CXR_dataset [49]		3360/840
Covid & Viral Pneumonia Classification	Covid-19 Radiography Dataset [50]		16930/4235
Retinal Blood Vessel Segmentation	Drive [51]	Color fundus	20/20
	Stare [52]		16/4
	Chase_DB1 [53]		22/6
Colon-Polyp Segmentation	KVASIR_SEG [54]	Endoscopy	800/200
	CVC_ClinicDB [55]		489/123
Optic-cup & Optic-disc Segmentation	CRFO [56]	Color fundus	35/9
	Drishti [57]		80/21
	G1020 [58]		632/159
	Origa [59]		520/130
	Papila [60]		390/98
Optic-cup, Optic-disc Segmentation & Glaucoma Classification	Refuge1 [61]		640/160
Glaucoma Classification	Rim-One [62]		425/107
Malaria Classification	NIH Malaria Dataset [63]	Microscopy images	11023/2756
Leukemia Classification	C-NMC_Leukemia [64]		8528/2133

depending on the dataset size and GPU memory availability—typically, a batch size of 32 was used when the training set included fewer than 1000 images, and 128 for larger datasets. Weighted cross-entropy loss was used as the classification loss function. As with segmentation, real-time data augmentation (e.g., flipping) was employed. Minimal preprocessing was used to maintain pipeline simplicity: all images were resized to 256×256, and histogram equalization was applied to enhance contrast.

4.3. Hyperparameter tuning

To optimize training efficiency and model performance, we tuned several hyperparameters. For segmentation, we used the Adam optimizer with an initial learning rate of 0.001. A cosine annealing learning rate scheduler with periodic restarts was employed to dynamically adjust the learning rate. The scheduler was configured with $T_0 = 1$ (first cycle = 1 epoch) and $T_{\text{mult}} = 2$, doubling the cycle length at each stage (e.g., 1, 2, 4, 8 epochs). The minimum learning rate was set to $\eta_{\min} = 1 \times 10^{-9}$. Early stopping was based on validation loss, with a patience of 30 epochs and a minimum improvement threshold of 1×10^{-10}. The best model was saved via checkpointing based on improvements in the validation Dice score. For classification, we also used the Adam optimizer with an initial learning rate of 0.001. A custom learning rate scheduler was applied, following a three-phase policy: (1) a linear warm-up phase over 25 epochs (from lr_{start} to $lr_{\max}$), (2) an optional sustain phase, and (3) an exponential decay phase toward $lr_{\min}$. This strategy stabilized early training and improved convergence. Early stopping with a 30-epoch patience was used based on validation loss. The best-performing model (based on validation accuracy) was retained via checkpointing.

4.4. LLM selection

In MedPrompt, the LLM serves as a workflow orchestrator—interpreting natural language prompts and generating structured task sequences, rather than performing image analysis itself. Its role is critical for enabling flexible, multi-step execution across classification and segmentation tasks. We used LlaMA-4-Maverick-17B, an open-source LLM known for strong instruction-following capabilities. No fine-tuning was required; we employed few-shot prompting with 10 curated examples. These examples included structured reasoning steps and did not overlap with the evaluation set. Inference was performed via Groq APIs.

4.5. Machine configuration

All experiments were conducted using Kaggle's GPU-accelerated platform. Specifically, we used a Kaggle Notebook instance equipped with two NVIDIA Tesla T4 GPUs, each offering 16 GB of GDDR6 memory and 2560 CUDA cores.

5. Experimental result analysis

This section presents the evaluation results of the proposed MedPrompt framework. The evaluation is divided into three parts: (i) structured output generation accuracy, (ii) performance of the DeepFusionLab model, and (iii) end-to-end framework evaluation. Because structured output generation and framework-level correctness are closely coupled, they are reported together in a single table.

5.1. DeepFusionLab performance across datasets

The DeepFusionLab architecture supports both classification and segmentation, and its performance was evaluated across a range of datasets for both task types. Each experiment was repeated five times, and the mean and standard deviation of the test metrics were computed to ensure robust performance assessment.

5.1.1. Evaluation on classification datasets

Table 4 presents the classification performance of DeepFusionLab across seven targets using eight distinct datasets (ten in total, including combinations). Results are reported as mean ±standard deviation, computed over five runs per dataset. From the results in Table 4, it is evident that DeepFusionLab performs well across a wide range of classification datasets. However, performance on smaller datasets such as Shenzhen and Rim-One was comparatively lower, primarily due to the limited number of training samples. Additionally, training time varied depending on the dataset, largely influenced by the behavior of early stopping. On high-quality datasets like CXR_Dataset, the model converged quickly, triggering early stopping and reducing training time. In contrast, for datasets with lower image quality, convergence was slower, leading to longer training durations.

Table 4
Evaluation of DeepFusionLab across classification datasets.

Target	Dataset	Accuracy	Precision	Recall	F1	AUC	Training time (min)
Tuberculosis	CXR_Dataset	0.9995 ± 0.0005	0.9924 ± 0.0005	0.9922 ± 0.0024	0.9923 ± 0.0023	1.0000 ± 0.0000	7.83 ± 1.5297
	Shenzhen	0.8972 ± 0.0197	0.8993 ± 0.0182	0.8975 ± 0.0196	0.8971 ± 0.0198	0.9442 ± 0.0122	2.7033 ± 0.8571
	Shenzhen + Montgomery	0.8985 ± 0.0127	0.9008 ± 0.0100	0.8991 ± 0.0123	0.8984 ± 0.0128	0.9434 ± 0.0029	2.6167 ± 0.0939
	Combined	0.9825 ± 0.0015	0.9804 ± 0.0026	0.9687 ± 0.0017	0.9744 ± 0.0021	0.9986 ± 0.0002	16.9000 ± 2.0500
Malaria	NIH Malaria Dataset	0.9776 ± 0.0006	0.9776 ± 0.0006	0.9776 ± 0.0006	0.9776 ± 0.0006	0.9969 ± 0.0004	72.7433 ± 9.5022
Leukemia	C-NMC_Leukemia	0.9808 ± 0.0013	0.9802 ± 0.0012	0.9754 ± 0.0023	0.9777 ± 0.0015	0.9959 ± 0.0008	63.8567 ± 7.4647
Glaucoma	REFUGE1	0.9750 ± 0.0102	0.9649 ± 0.0354	0.8935 ± 0.0312	0.9249 ± 0.0301	0.9630 ± 0.0166	4.5700 ± 0.1981
	RIM-ONE	0.9031 ± 0.0051	0.8911 ± 0.0050	0.9023 ± 0.0072	0.8960 ± 0.0058	0.9628 ± 0.0009	10.5000 ± 5.4700
	Combined	0.9363 ± 0.0070	0.9214 ± 0.0132	0.9026 ± 0.0091	0.9112 ± 0.0093	0.9668 ± 0.0013	19.8667 ± 4.3586
COVID	COVID-19 Radiography Database	0.9969 ± 0.0005	0.9967 ± 0.0005	0.9952 ± 0.0007	0.9959 ± 0.0006	0.9996 ± 0.0004	90.6533 ± 12.4130
Viral Pneumonia	COVID-19 Radiography Database	0.9958 ± 0.0002	0.9906 ± 0.0008	0.9890 ± 0.0008	0.9898 ± 0.0005	0.9998 ± 0.0001	95.7233 ± 16.1508
COVID, Viral Pneumonia, Lung Opacity	COVID-19 Radiography Database	0.9580 ± 0.0021	0.9671 ± 0.0018	0.9619 ± 0.0013	0.9645 ± 0.0013	N/A	112.3000 ± 10.4539

5.1.2. Evaluation on segmentation datasets

Table 5 presents the segmentation performance of DeepFusionLab, reported as mean and standard deviation, across five segmentation targets using 19 individual datasets (24 in total, including combined datasets). From the results in Table 5, it is evident that DeepFusionLab also performs effectively across a wide range of segmentation datasets. However, performance on datasets with a very limited number of images — such as those related to retinal blood vessel segmentation — was slightly lower compared to larger datasets. Training time varied depending on both dataset size and image quality, as well as the triggering of early stopping. For high-quality images, the model converged quickly, allowing early stopping to activate and reduce training time. In contrast, datasets with lower image quality required longer training to reach convergence, resulting in extended training durations.

5.2. Generalizability across diverse medical datasets & ablation study of DeepFusionLab

Medical imaging datasets often differ significantly in terms of size, resolution, modality, and overall visual quality, presenting challenges for models aiming to generalize across multiple domains. DeepFusionLab was specifically designed with these variations in mind. The inclusion of the ASPP module allows the model to capture multi-scale contextual information efficiently without a substantial increase in parameters. This is especially beneficial for datasets with subtle or spatially distributed disease patterns. The encoder extracts both low-level spatial features and high-level semantic representations, enabling effective adaptation to domain-specific differences such as chest X-rays versus fundus images.

The MFF module enhances generalization by integrating features across multiple receptive fields, improving robustness to variability in lesion size, shape, and noise. Meanwhile, the CAFSE module refines decoder outputs by combining global semantic information with local edge-level detail, a feature particularly useful for handling datasets with inconsistent resolutions or poorly delineated boundaries. CAFSE's use of 1×1 convolutions for channel reduction, global average pooling for spatial compression, and lightweight decoding paths enables DeepFusionLab to balance expressiveness with computational efficiency, facilitating deployment across datasets with varying sizes and hardware constraints. Table 6 presents the results of our ablation study, where different architectural components were selectively removed to evaluate their individual contributions. Ablation was performed on the combined segmentation datasets, as the classification pathway contains no additional modules.

From the results of the ablation study, we observe that DeepFusionLab performs robustly across both the lung and retinal blood vessel datasets. Notably, the retinal blood vessel dataset contains only 58 training images. In this low-data scenario, the introduction of the CBAM attention module led to overfitting, ultimately reducing performance. Consequently, CBAM was excluded from the final architecture. For datasets with larger training sizes, the proposed model consistently outperformed its simplified variants, confirming the effectiveness of the included architectural components. However, even in these cases, the inclusion of CBAM did not yield meaningful performance improvements, further justifying its exclusion. We also evaluated the impact of using different encoder backbones, as presented in Table 7. While the MobileNet-V2 encoder results in a lighter model, it significantly underperforms compared to other alternatives. Some performance gains were observed with encoders like ResNet-50 and DenseNet-121 on certain datasets. However, EfficientNet-B1 consistently performed equal to or better than EfficientNet-B0 across all target domains. Despite this, the marginal improvements offered by these larger encoders did not justify their increased computational cost. Based on these findings, we selected an encoder architecture that provides an optimal balance

Table 5
Evaluation of DeepFusionLab across segmentation datasets.

Target	Dataset	Dice	IoU	AUC	Accuracy	Precision	Recall	Training time (min)
Lung	JSRT	0.9847 ± 0.0006	0.9711 ± 0.0010	0.9970 ± 0.0004	0.9876 ± 0.0004	0.9842 ± 0.0001	0.9864 ± 0.0007	8.0940 ± 1.0387
	Montgomery	0.9829 ± 0.0005	0.9683 ± 0.0006	0.9968 ± 0.0004	0.9883 ± 0.0009	0.9851 ± 0.0007	0.9827 ± 0.0007	6.2220 ± 0.5472
	Combined	0.9856 ± 0.0003	0.9724 ± 0.0003	0.9959 ± 0.0007	0.9887 ± 0.0004	0.9858 ± 0.0004	0.9862 ± 0.0003	14.2333 ± 2.4670
Retinal Blood Vessel	Drive	0.7967 ± 0.0094	0.7018 ± 0.0033	0.9657 ± 0.0011	0.9311 ± 0.0060	0.7783 ± 0.0087	0.8339 ± 0.0061	5.3380 ± 0.3828
	STARE	0.8001 ± 0.0057	0.7100 ± 0.0046	0.9780 ± 0.0007	0.9439 ± 0.0034	0.7931 ± 0.0154	0.8296 ± 0.0125	4.2500 ± 0.5622
	CHASE_DB1	0.8209 ± 0.0041	0.7352 ± 0.0031	0.9820 ± 0.0010	0.9528 ± 0.0011	0.8090 ± 0.0029	0.8509 ± 0.0052	4.4133 ± 0.8749
	Combined	0.8106 ± 0.0024	0.7178 ± 0.0013	0.9717 ± 0.0009	0.9400 ± 0.0006	0.7965 ± 0.0030	0.8393 ± 0.0012	7.3600 ± 1.9200
Colon Polyp	Kvasir-SEG	0.9327 ± 0.0020	0.8923 ± 0.0029	0.9794 ± 0.0033	0.9694 ± 0.0015	0.9388 ± 0.0041	0.9435 ± 0.0036	22.6900 ± 3.4528
	CVC-ClinicDB	0.9605 ± 0.0035	0.9294 ± 0.0049	0.9934 ± 0.0007	0.9889 ± 0.0009	0.9619 ± 0.0056	0.9630 ± 0.0023	24.5100 ± 2.6780
	Combined	0.9451 ± 0.0012	0.9085 ± 0.0014	0.9829 ± 0.0010	0.9763 ± 0.0007	0.9530 ± 0.0016	0.9496 ± 0.0003	49.7700 ± 3.3788
Optic-cup	CRFO	0.8706 ± 0.0145	0.8202 ± 0.0114	0.9966 ± 0.0033	0.9957 ± 0.0010	0.8963 ± 0.0204	0.8879 ± 0.0306	12.5333 ± 2.9200
	DRISHTI	0.9485 ± 0.0045	0.9112 ± 0.0071	0.9990 ± 0.0008	0.9976 ± 0.0002	0.9451 ± 0.0063	0.9627 ± 0.0106	24.8533 ± 3.4808
	G1020	0.9187 ± 0.0023	0.8642 ± 0.0037	0.9959 ± 0.0001	0.9975 ± 0.0002	0.9254 ± 0.0003	0.9280 ± 0.0044	26.2067 ± 1.2349
	ORIGA	0.9306 ± 0.0017	0.8826 ± 0.0026	0.9954 ± 0.0024	0.9969 ± 0.0011	0.9413 ± 0.0079	0.9352 ± 0.0043	25.0400 ± 2.4802
	PAPILA	0.8823 ± 0.0035	0.8236 ± 0.0035	0.9947 ± 0.0003	0.9965 ± 0.0001	0.9020 ± 0.0039	0.8948 ± 0.0059	29.7567 ± 2.5762
	REFUGE1	0.9430 ± 0.0027	0.8997 ± 0.0039	0.9969 ± 0.0018	0.9979 ± 0.0008	0.9464 ± 0.0051	0.9482 ± 0.0172	31.5867 ± 3.3564
	Combined	0.9252 ± 0.0020	0.8763 ± 0.0029	0.9976 ± 0.0011	0.9974 ± 0.0009	0.9265 ± 0.0038	0.9402 ± 0.0020	70.3667 ± 14.6573
Optic-Disc	CRFO	0.9772 ± 0.0036	0.9597 ± 0.0062	0.9998 ± 0.0008	0.9986 ± 0.0002	0.9818 ± 0.0049	0.9768 ± 0.0101	9.7033 ± 0.9666
	DRISHTI	0.9842 ± 0.0009	0.9707 ± 0.0013	0.9998 ± 0.0004	0.9984 ± 0.0001	0.9838 ± 0.0016	0.9863 ± 0.0154	20.7533 ± 2.6540
	G1020	0.9667 ± 0.0032	0.9394 ± 0.0049	0.9987 ± 0.0001	0.9979 ± 0.0002	0.9663 ± 0.0049	0.9700 ± 0.0014	27.3033 ± 7.3422
	ORIGA	0.9805 ± 0.0020	0.9628 ± 0.0040	0.9955 ± 0.0057	0.9963 ± 0.0037	0.9792 ± 0.0055	0.9828 ± 0.0015	24.7867 ± 4.2306
	PAPILA	0.9762 ± 0.0006	0.9558 ± 0.0008	0.9985 ± 0.0001	0.9973 ± 0.0001	0.9723 ± 0.0024	0.9825 ± 0.0014	32.6200 ± 7.6245
	REFUGE1	0.9791 ± 0.0004	0.9608 ± 0.0004	0.9993 ± 0.0000	0.9986 ± 0.0001	0.9787 ± 0.0017	0.9814 ± 0.0013	35.4500 ± 3.0106
	Combined	0.9775 ± 0.0005	0.9579 ± 0.0008	0.9990 ± 0.0008	0.9984 ± 0.0001	0.9768 ± 0.0008	0.9800 ± 0.0001	146.1033 ± 8.9708

between computational efficiency and predictive performance. The proposed design ensures strong generalization across both small and large datasets while avoiding unnecessary architectural complexity.

To optimize the performance of different architectural blocks, we tuned the hyperparameters of the MFF and CAFSE blocks. The corresponding results are presented in Table 8. From the table, it is evident that as the number of parameters increases, performance generally improves—except for the SE reduction ratio on the combined optic disc dataset, where a slight degradation was observed. To determine the most optimal hyperparameter values, we selected configurations that offered the best trade-off between performance and efficiency. For instance, increasing the coarse kernel size from 3 to 5 yielded only a marginal performance gain (~0.1%) while increasing the parameter count by over 150%, which was not justifiable in terms of efficiency.

5.3. Overall framework performance

The proposed MedPrompt framework uniquely integrates natural language understanding, dynamic task planning, and modular CNN-based execution (via DeepFusionLab) to enable flexible, user-driven medical image analysis. Unlike prior approaches that address isolated tasks or require full model retraining for new tasks, MedPrompt facilitates scalable, prompt-driven multi-task inference. To assess its system-level capabilities, we evaluated MedPrompt using metrics that measure both interpretive accuracy and practical usability:

Table 6
Ablation study.

Model & No of Parameters	Combined dataset and number of training dataset images									
	Lung		RetinalBlood Vessel		ColonPolyp		Optic-Cup		Optic-Disc	
	307		58		1289		2297		2481	
	Dice	IoU	Dice	IoU	Dice	IoU	Dice	IoU	Dice	IoU
EfficientNet B0 + Decoder (4.1M)	0.981	0.963	0.792	0.696	0.906	0.865	0.871	0.803	0.925	0.908
EfficientNet B0 + ASPP + Decoder (6.2M)	0.982	0.969	0.799	0.699	0.910	0.869	0.874	0.806	0.929	0.920
EfficientNet B0 + ASPP + Decoder + MFF (6.8M)	0.984	0.971	0.806	0.703	0.929	0.882	0.882	0.830	0.933	0.921
Proposed Model + CBAM (8.13M)	0.986	0.972	0.807	0.707	0.945	0.909	0.927	0.877	0.979	0.959
Proposed Model with EfficientNet B1 (10.2M)	0.987	0.974	0.821	0.718	0.950	0.909	0.931	0.879	0.979	0.961
Proposed Model with ResNet 50 (29.02M)	0.973	0.968	0.805	0.707	0.942	0.9	0.927	0.878	0.968	0.955
Proposed Model with DenseNet 121 (10.6M)	0.983	0.971	0.808	0.709	0.946	0.910	0.930	0.879	0.97	0.952
Proposed Model with MobileNet v2 (6.33M)	0.942	0.939	0.769	0.681	0.924	0.882	0.894	0.855	0.925	0.911
Proposed Model (8.12M)	0.986	0.973	0.811	0.718	0.945	0.909	0.925	0.877	0.978	0.958

Table 7
Encoder or backbone parameter comparison.

Backbone or Encoder	Parameter count of the Encoder & Modified DeepFusionLab	GFLOPs	Combined dataset of target organ									
			Lung		Retinal Blood Vessel		Colon Polyp		Optic Cup		Optic Disc	
			Dice	IoU	Dice	IoU	Dice	IoU	Dice	IoU	Dice	IoU
EfficientNet B0 (Proposed)	4.007M & 8.12M	6.44	0.986	0.973	0.811	0.718	0.945	0.909	0.925	0.877	0.978	0.958
EfficientNetB1	6.087M & 10.2 M	6.69	0.987	0.974	0.821	0.718	0.950	0.909	0.931	0.879	0.979	0.961
ResNet50	23.508M & 29.02M	14.74	0.973	0.968	0.805	0.707	0.942	0.9	0.927	0.878	0.968	0.955
MobileNetv2	2.223M & 6.33M	2.00	0.942	0.939	0.769	0.681	0.924	0.882	0.894	0.855	0.925	0.911
DenseNet121	6.954M & 10.6M	5.35	0.983	0.971	0.808	0.709	0.946	0.910	0.930	0.879	0.97	0.952

Table 8
Performance comparison across different blocks and hyperparameters.

Block	Hyperparameter	Value	Parameters of the block	Combined dataset of target organ									
				Lung		Retinal blood vessel		Colon Polyp		Optic Cup		Optic Disc	
				Dice	IoU	Dice	IoU	Dice	IoU	Dice	IoU	Dice	IoU
MFF	Channel Projection Ratio	0.7	484K	0.976	0.965	0.796	0.704	0.936	0.901	0.914	0.868	0.970	0.946
		0.9	617K	0.984	0.970	0.807	0.717	0.942	0.906	0.923	0.875	0.975	0.955
		1 (Proposed)	684K	**0.986**	0.973	0.811	**0.718**	0.945	0.909	0.925	**0.877**	**0.978**	0.958
		1.2	817K	0.986	**0.974**	**0.812**	0.718	**0.946**	**0.910**	**0.926**	**0.877**	0.978	**0.959**
	SE Reduction Ratio	4:1	701K	**0.988**	0.973	0.811	**0.719**	0.945	0.909	0.925	**0.878**	0.976	0.957
		8:1 (Proposed)	684K	0.986	**0.973**	**0.811**	0.718	**0.945**	**0.909**	**0.925**	0.877	0.978	0.958
		16:1	677K	0.983	0.971	0.807	0.713	0.943	0.906	0.921	0.875	**0.980**	**0.959**
CAFSE	Coarse Kernel Size	1	132K	0.982	0.971	0.804	0.713	0.940	0.904	0.920	0.874	0.972	0.954
		3 (Proposed)	656K	0.986	**0.973**	0.811	**0.719**	**0.945**	**0.909**	0.925	0.877	0.978	0.958
		5	1705K	**0.987**	0.973	**0.812**	0.719	**0.945**	**0.909**	**0.926**	**0.879**	**0.979**	**0.959**
	Fine Kernel Size	1	656K	0.986	0.973	**0.811**	0.719	0.945	**0.909**	**0.925**	0.877	0.978	0.958
		3 (Proposed)	1181K	**0.987**	**0.974**	0.811	**0.720**	**0.946**	0.909	0.925	**0.878**	**0.980**	**0.961**

- Intention Correctness: Assesses whether the LLM accurately identifies the task intent (i.e., classification or segmentation).
- Target Correctness: Measures the LLM's ability to correctly identify and normalize the anatomical region or disease mentioned in the user prompt (e.g., interpreting ''pulmonary parenchyma'' as ''lung'').
- Weight Correctness: Evaluates whether the framework correctly selects the most appropriate pretrained model weight. If no suitable weight exists, the system should avoid assigning an arbitrary one.
- Condition and Dependency Correctness: Captures the system's ability to detect logical dependencies among tasks (e.g., performing lung segmentation only if pneumonia is detected).
- Overall Correctness: Reflects whether the final output of the pipeline accurately and coherently satisfies the user's query.

These five metrics collectively evaluate the correctness of the JSON structure generated by the LLM and the final system output. In our experiments, we found that when the generated JSON included accurate intent, target, and dependency information, the final outputs

Table 9
Performance of MedPrompt.

Question type	Number of Questions	Correctly identified					Overall correctness	Average Time (CPU)
		Intention	Target	Weight	Condition	Dependency		
Simple	50	50/50 100%	49/50 98%	49/50 98%	Null	Null	49/50 98%	2.11s
Complex	50	114/115 99.13%	114/115 99.13%	112/115 97.39%	57/60 95%	57/60 95%	111/115 96.52%	2.77s
Overall	100	164/165 99.39%	163/165 98.79%	161/165 97.58%	57/60 95%	57/60 95%	160/165 96.97%	2.44s

were consistently correct. Hence, these metrics are reported alongside overall results rather in Table 9. We also included average time in Table 9 by measuring the average latency of the full pipeline — covering LLM inference, weight selection, and task execution — to evaluate suitability for real-time or interactive use. Framework latency was measured on Kaggle's CPU-based environment, and the reported times reflect end-to-end response generation under CPU constraints.

For evaluation, we considered both simple and complex user prompts. Simple prompts refer to queries that involve a single, independent task without any conditional dependencies. For example, users may request: "Delimit the pulmonary parenchyma region from the provided thoracic radiograph", "Evaluate this chest radiograph for indications of *pulmonary Mycobacterium tuberculosis* infection", or "Examine this hematologic image for indications of *acute lymphoblastic leukemia*". These types of queries are straightforward and were consistently handled with high accuracy by the MedPrompt framework.

In contrast, complex prompts involve multiple tasks, conditions, and dependencies that require the system to understand and sequence actions based on prior outcomes. Examples of such queries include: "Assess for viral etiology of pneumonia. Upon confirmation, segment pulmonary structures for severity quantification" (simplified: Check for viral pneumonia, and if confirmed, segment the lungs for further assessment); "Determine if SARS-CoV-2 pneumonia is present. If confirmed, assess for concurrent Mycobacterial pulmonary involvement" (simplified: Classify for COVID-19, and if confirmed, check for co-existing tuberculosis); and "From this retinal fundus image, assess for optic neuropathy due to elevated intraocular pressure. If found, delineate the optic cup to support diagnosis" (simplified: Check for glaucoma, and if detected, segment the optic cup).

Our evaluation shows that the framework handles simple queries with near-perfect accuracy. The small percentage of incorrect predictions — approximately 3% — were primarily associated with complex, condition-dependent queries. One example where the LLM-generated JSON failed was: "Assess this thoracic radiograph for parenchymal opacification. If inconclusive, screen for viral lower respiratory tract infection". In this case, the ambiguity surrounding the "inconclusive" outcome led to a breakdown in task sequencing.

Despite such edge cases, MedPrompt demonstrated strong overall performance in both understanding and executing a variety of prompt types, highlighting its effectiveness for real-world clinical deployment. The system is designed to robustly handle multi-step execution plans produced by the LLM. Furthermore, when the LLM determines that a task falls outside its knowledge scope or the available model weight repository, it explicitly instructs the system to skip the unsupported operation—preventing erroneous or arbitrary execution.

5.4. Impact of LLM selection

Table 10 presents the accuracy and latency comparison of various LLMs integrated within the MedPrompt framework. Unlike end-to-end medical reasoning systems, where domain-specific models like Lingshu excel, the role of the LLM in MedPrompt is fundamentally different. As illustrated in Fig. 8, MedPrompt adopts a modular design where the LLM is solely responsible for deterministic prompt parsing and workflow generation. It does not perform diagnostic reasoning; rather, the medical expertise resides in the pretrained CNNs, such as DeepFusionLab, which execute the actual image analysis. This architectural separation makes domain-specific LLMs unnecessary for our framework, as their strengths in multimodal reasoning and report generation are not leveraged here. Our experiments confirm that the choice of LLM primarily impacts response latency, not output correctness. Without external knowledge base integration, prompt parsing averaged 1 s. When ontology-based target normalization was enabled, latency increased—LLaMA-4 averaged 2.5 s, while Gemma-2 and Qwen varied between 2–5 s. Since the task centers on structured output generation from constrained prompts — not creative or open-ended generation — all tested LLMs (LLaMA, Gemma, Qwen) delivered comparable accuracy. In practical clinical workflows, physicians use consistent, domain-standard terminology, reducing reliance on large, knowledge-rich models. Hence, lightweight LLMs — or even rule-based systems — may be more suitable for latency-critical environments like real-time diagnostics.

5.5. Comparison

To the best of our knowledge, MedPrompt introduces a fundamentally novel approach by combining natural language understanding with dynamic model routing for medical image analysis. This holistic framework has no direct equivalent in existing literature, making head-to-head comparisons with prior systems infeasible or potentially misleading. For example, models like MedSAM, MedCLIP-SAM, and BioMedGPT require spatial prompts or structured inputs, which differ significantly from MedPrompt's use of free-form natural language instructions. Moreover, these models do not support integrated classification and segmentation workflows. Since MedPrompt delegates image processing to DeepFusionLab, it is more appropriate to compare DeepFusionLab with other CNN-based models. Therefore, to ensure a meaningful and fair evaluation, we benchmark DeepFusionLab — the core vision module within MedPrompt — against state-of-the-art classification and segmentation models. This approach allows us to rigorously assess performance across both task types.

5.5.1. Comparison of DeepFusionLab

DeepFusionLab is designed to support both classification and segmentation tasks. To evaluate its effectiveness, we compare its performance on each task separately. Table 11 presents the classification results of DeepFusionLab in comparison with existing models across multiple datasets. Table 12 compares DeepFusionLab with other studies over segmentation datasets.

Due to the lack of publicly available source code and unreported standard deviations in most comparative works, we evaluated statistical significance using the proportion z-test for results in Tables 11 and 12. The proportion z-test is highly sensitive to the number of samples; consequently, small test sets can yield high p-values even when noticeable performance gaps exist. For instance, in the REFUGE-1 dataset, our method achieved a 96.08% score, whereas a baseline model with a 3% lower score resulted in a p-value of 0.258, primarily due to the test set size of only 160 images. This limitation is even more

Table 10
Comparison of LLM performance.

LLM	Workflow accuracy	Average latency	Use case suitability
Llama-4 17B	97%	2.5 s	Optimal for structured parsing
Gemma-2 9B	97%	3.2 s	Similar accuracy, higher latency
Qwen 32B	96%	4.1 s	Marginal performance difference

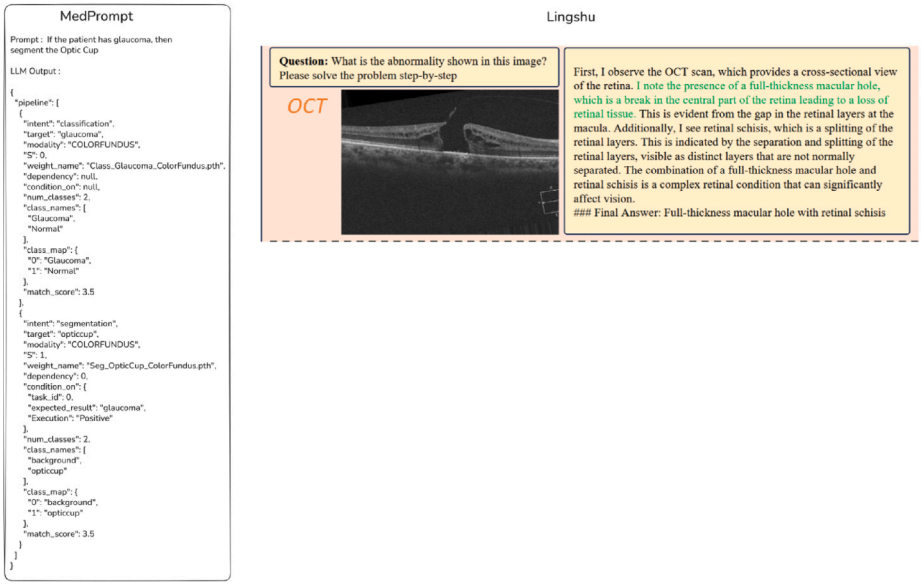


Fig. 8. The role of LLM in MedPrompt vs Lingshu question-answering.

Table 11
Comparison over classification dataset.

Objective	Dataset	Work	Accuracy (%)	F1 (%)	p-value (Acc)	p-value (F1)
Tuberculosis	CXR_Dataset	Rahman et al. [49]	98.60	–	3.1978e−02	–
		Huy et al. [65]	98.80	98.80	4.9272e−02	3.7222e−01
		This study	**99.95**	**99.23**	–	–
	Shenzhen	Rajaraman et al. [66]	88.79	88.73	8.0654e−01	7.9665e−01
		This study	**89.72**	**89.71**	–	–
Malaria	NIH Malaria Dataset	Boit et al. [67]	97.68	97.65	8.4230e−01	7.8509e−01
		Madhu et al. [68]	**99.35**	**99.36**	7.5779e−07	6.1786e−07
		This study	97.76	97.76	–	–
Leukemia	C_NMC_Leukemia	Jawahar et al. [69]	91.98	96.00	4.8880e−20	8.7681e−04
		Sampathila et al. [70]	95.54	95.43	2.3563e−06	2.4795e−05
		This study	**98.08**	**97.77**	–	–
Glaucoma	Rim-One	Shoukat et al. [71]	**96.15**	**97.00**	9.2123e−02	3.1990e−02
		This study	90.31	89.60	–	–
	Refuge1	Neto et al. [72]	97.00	–	7.8449e−01	–
		This study	**97.50**	**92.49**	–	–
COVID, Viral Pneumonia, Lung Opacity	COVID-19 Radiography Dataset	Yen et al. [73] (3 Class Classification)	97.03	97.03	–	–
		Chauhan et al. [74]	**98.91**	**97.86**	4.7367e−19	9.5154e−05
		This study (All Classes)	95.80	96.45	–	–

pronounced in datasets like DRIVE and STARE, which include fewer than 20 images. In contrast, classification datasets with larger test sets produced statistically significant differences.

We also performed statistical significance testing of DeepFusionLab against other state-of-the-art models (UNet, UNet++, DeepLabv3+), with results summarized in Table 13. This analysis employed both paired t-tests and one-way ANOVA. The paired t-test yielded positive t-values in 27 out of 30 comparisons, indicating that DeepFusionLab outperformed baseline models in most cases. Negative t-values appeared only in three instances — specifically, the Dice and IoU scores for retinal blood vessel segmentation and the IoU score for optic disc segmentation, all compared with UNet++ — indicating marginally lower performance in those specific tasks. In all other scenarios, DeepFusionLab demonstrated superior results. The p-values shown in Table 13 were computed using the two-sample Welch's t-test, which was applied wherever both mean and standard deviation were available.

Table 12
Comparison over segmentation datasets.

Objective	Dataset	Work	IoU (%)	Dice (%)	p-value (IoU)	p-value (Dice)
Lung	JSRT	Liu et al. [75]	95.80	97.90	7.2318e−01	8.3094e−01
		Yuan et al. [76]	94.90	97.40	5.7260e−01	7.0676e−01
		This study	**97.11**	**98.47**	–	–
	Montgomery	Liu et al. [75]	95.50	97.40	8.0041e−01	8.1641e−01
		Yuan et al. [76]	96.10	98.00	8.8744e−01	9.3361e−01
		This study	**96.80**	**98.30**	–	–
Retinal Blood Vessel	Drive	Liu et al. [77]	–	83.30	–	7.6759e−01
		Rui et al. [78]	**72.34**	**83.94**	8.8003e−01	7.2634e−01
		This study	70.18	79.67	–	–
	STARE	Liu et al. [77]	–	**86.60**	–	8.0239e−01
		Rui et al. [78]	**73.53**	85.20	9.3630e−01	8.4619e−01
		This study	71.00	80.00	–	–
	CHASE_DB1	Liu et al. [77]	–	81.60	–	9.8243e−01
		Rui et al. [78]	70.59	80.40	9.0996e−01	9.4022e−01
		This study	**73.52**	**82.09**	–	–
Colon-Polyp	Kvasir-SEG	Xue et al. [79]	89.10	**93.90**	9.6664e−01	7.9708e−01
		Li et al. [80]	86.30	91.90	3.7125e−01	6.0106e−01
		This study	**89.23**	93.27	–	–
	CVC-ClinicDB	Xue et al. [79]	90.20	94.70	4.3929e−01	6.1421e−01
		Li et al. [80]	88.70	93.90	2.4950e−01	4.4023e−01
		This study	**92.94**	**96.05**	–	–
Optic-Cup	DRISHTI	Guo et al. [81]	–	**92.70**	–	5.4453e−01
		This study	**82.02**	87.06	–	–
	REFUGE1	Yi et al. [82]	–	90.82	–	2.3558e−01
		Bhattacharya et al. [83]	–	93.85	–	8.6463e−01
		This study	**89.97**	**94.30**	–	–
Optic-Disc	DRISHTI	Guo et al. [81]	96.57	98.24	9.2643e−01	9.6370e−01
		Bhattacharya et al. [83]	94.37	97.10	6.6556e−01	7.7255e−01
		This study	**97.07**	**98.42**	–	–
	ORIGA	Raza et al. [84]	88.00	91.00	1.3117e−02	1.2472e−02
		This study	**96.28**	**98.05**	–	–
	REFUGE1	Bhattacharya et al. [83]	93.23	96.48	2.5709e−01	4.3856e−01
		Guo et al. [81]	95.80	–	8.9902e−01	–
		This study	**96.08**	**97.91**	–	–

In contrast, the proportion z-test was used for Tables 11 and 12 due to the absence of standard deviation data in most related works.

All p-values reported in Table 13 were below 0.05, confirming that the observed performance differences — whether favorable or marginally unfavorable — are statistically significant and unlikely to be due to chance. Furthermore, the one-way ANOVA test conducted across all datasets and metrics yielded extremely small p-values ($p \ll 0.05$), indicating statistically significant performance differences between the models (UNet, UNet++, DeepLabv3+, and DeepFusionLab) across all evaluated organs and metrics. While some individual dataset comparisons yielded mixed significance levels — mainly due to small sample sizes in segmentation datasets — our comprehensive evaluation in Table 13 demonstrates statistically significant improvements across all metrics and organ-specific tasks ($p < 0.05$ in all cases).

5.5.2. Discussion on comparison

As a unified model designed to perform both classification and segmentation across a diverse range of medical imaging datasets, DeepFusionLab operates with a fixed input shape of 256×256. However, many publicly available datasets, such as DRIVE and STARE, use higher-resolution inputs (e.g., 565×584 and 700×605, respectively). Due to this resolution discrepancy, DeepFusionLab may not achieve state-of-the-art performance on these datasets compared to models specifically tuned for their native input sizes. Nevertheless, the model still demonstrates robust and competitive results. Further improvements, such as incorporating additional architectural blocks or using a deeper encoder like EfficientNet-B2, may enhance performance—particularly on large-scale datasets such as the COVID-19 Radiography Dataset. However, these changes would increase the number of trainable parameters, potentially introducing overfitting on smaller datasets like DRIVE and STARE. Considering these trade-offs, we conclude that DeepFusionLab delivers a balanced performance across a wide variety of datasets while maintaining a reasonable model size, demonstrating its adaptability and generalization capabilities. Note that MedPrompt is not directly comparable to models like MedSAM due to fundamental differences in input modality and system design. While MedSAM relies on spatial prompts, MedPrompt operates on natural language instructions, enabling integrated classification and conditional segmentation. Its modular architecture supports dynamic routing of task-specific weights, offering real-time execution and scalable task extensibility without retraining. Rather than pursuing generalization, MedPrompt leverages specialized CNNs for superior, task-specific performance.

5.6. Demonstration

To illustrate the effectiveness of the proposed MedPrompt pipeline, we showcase two core outcomes: the structured JSON output generated by the LLM, and the corresponding system response. Fig. 9 displays a sample of the structured output generated from a natural language prompt. The structured output highlights the LLM's ability to accurately interpret complex user instructions and decompose them

Table 13
DeepFusionLab comparison with SOTA models.

Model	Metric	Combined dataset of target organ									
		Lung		Retinal BloodVessel		Colon Polyp		Optic Cup		Optic Disc	
		Dice	IoU	Dice	IoU	Dice	IoU	Dice	IoU	Dice	IoU
UNet	Score	0.9613 ± 0.0027	0.9240 ± 0.0005	0.8024 ± 0.001	0.7109 ± 0.0005	0.8917 ± 0.0002	0.8876 ± 0.0018	0.8605 ± 0.0007	0.8253 ± 0.0005	0.9457 ± 0.0019	0.9298 ± 0.0011
	p-value	3.735e−03	2.450e−07	1.616e−02	5.794e−03	1.152e−04	1.361e−04	7.465e−05	8.009e−04	6.134e−04	8.735e−06
UNet++	Score	0.9672 ± 0.0003	0.9365 ± 0.0011	0.8211 ± 0.0007	0.7213 ± 0.001	0.9018 ± 0.0009	0.8931 ± 0.0017	0.8820 ± 0.0004	0.8333 ± 0.001	0.9687 ± 0.0012	0.9620 ± 0.0005
	p-value	1.882e−07	1.266e−04	1.180e−02	2.338e−02	2.195e−06	3.272e−04	4.651e−04	5.314e−04	2.249e−03	3.262e−03
Deep Labv3+	Score	0.9823 ± 0.0006	0.9688 ± 0.0013	0.799 ± 0.0024	0.699 ± 0.0012	0.910 ± 0.0003	0.869 ± 0.0004	0.874 ± 0.0005	0.806 ± 0.0021	0.929 ± 0.0009	0.920 ± 0.0007
	p-value	3.660e−03	3.518e−02	4.079e−03	5.372e−05	1.863e−04	1.630e−04	2.513e−04	1.073e−05	2.606e−06	5.086e−07
Deep Fusion Lab	Score	0.9856 ± 0.0003	0.9724 ± 0.0003	0.8106 ± 0.0024	0.7178 ± 0.0013	0.9451 ± 0.0012	0.9085 ± 0.0014	0.9252 ± 0.0020	0.8763 ± 0.0029	0.9775 ± 0.0005	0.9579 ± 0.0008



Fig. 9. Generated structured output by LLM.

into well-formed executable tasks. Fig. 10 further demonstrates MedPrompt's ability to process a user query and generate an accurate and interpretable answer.

6. Conclusion and future work

In this work, we introduced MedPrompt, a novel framework that integrates an LLM with a modular CNN to perform medical image segmentation and classification based on natural language instructions. By decoupling task planning from image analysis, MedPrompt allows users to describe their objectives in plain language while the system dynamically selects and applies the appropriate pretrained weights to execute the task. Our experiments, conducted on 19 publicly available datasets spanning 12 tasks and 5 imaging modalities, demonstrate that MedPrompt achieves high accuracy and low response latency, making it suitable for near real-time clinical applications. Its modular design also supports scalability: new tasks can be added by simply training and registering new model weights, without modifying the overall framework.

Currently, MedPrompt supports a focused set of 2D medical imaging tasks. In future work, we plan to extend support to 3D modalities such as CT and MRI, broaden the coverage of anatomical targets and disease

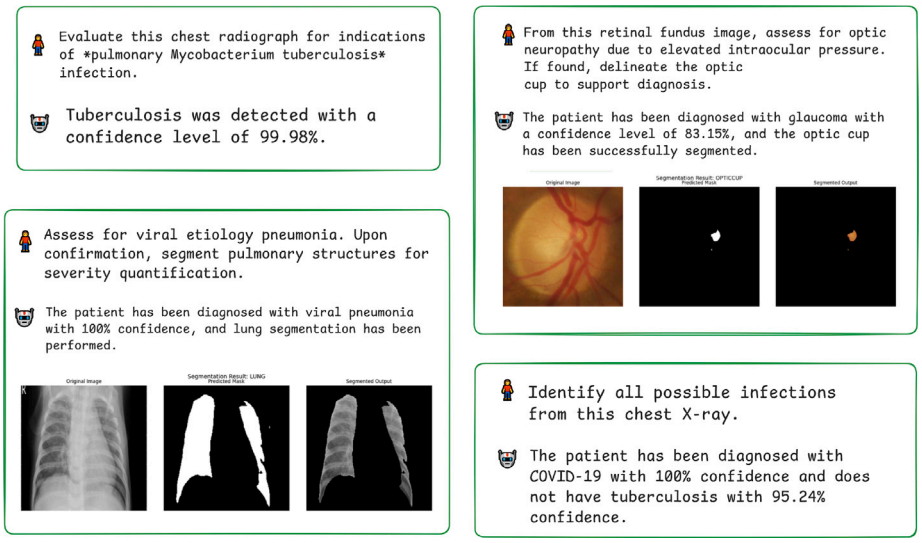


Fig. 10. Demonstration of MedPrompt's question-answering capability.

types, and explore model compression and quantization techniques for deployment in resource-constrained environments. Expanding to 3D modalities introduces challenges, including the high computational and memory demands of volumetric data and the need for specialized 3D backbone architectures. We intend to address these challenges through the development of lightweight volumetric CNN/Transformer architectures, hybrid slice–volume strategies, and efficient training paradigms. Overall, MedPrompt represents a significant step toward building intelligent, adaptable, and user-friendly medical AI systems capable of responding to natural language instructions and operating across a wide range of clinical imaging scenarios.

CRediT authorship contribution statement

Shadman Sobhan: Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Data curation, Conceptualization. **Kazi Abrar Mahmud:** Writing – original draft, Visualization, Validation, Methodology, Data curation, Conceptualization. **Abduz Zami:** Writing – original draft, Visualization, Methodology, Investigation, Data curation, Conceptualization. **Mohiuddin Ahmed:** Writing – review & editing, Visualization, Validation, Supervision, Software, Methodology, Investigation, Data curation, Conceptualization. **Md Palash Uddin:** Writing – review & editing, Visualization, Validation, Supervision, Software, Methodology, Investigation, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The links to the data have been included in the manuscript.

References

[1] R. Weissleder, M. Nahrendorf, Advancing biomedical imaging, Proc. Natl. Acad. Sci. 112 (47) (2015) 14424–14428.
[2] D.H. Ballard, K.R. Burton, N. Lakomkin, S. Kim, P. Rajiah, M.J. Patel, P. Mazaheri, G.J. Whitman, The role of imaging in health screening: screening for specific conditions, Academic Radiol. 28 (4) (2021) 548–563.
[3] A.W. Salehi, S. Khan, G. Gupta, B.I. Alabduallah, A. Almjally, H. Alsolai, T. Siddiqui, A. Mellit, A study of CNN and transfer learning in medical imaging: Advantages, challenges, future scope, Sustainability 15 (7) (2023) 5930.
[4] K. Suzuki, Overview of deep learning in medical imaging, Radiol. Phys. Technol. 10 (3) (2017) 257–273.
[5] D. Sarvamangala, R.V. Kulkarni, Convolutional neural networks in medical image understanding: a survey, Evol. Intell. 15 (1) (2022) 1–22.
[6] C.L. Chowdhary, D.P. Acharjya, Segmentation and feature extraction in medical imaging: a systematic review, Procedia Comput. Sci. 167 (2020) 26–36.
[7] R. Yang, Y. Yu, Artificial convolutional neural network in object detection and semantic segmentation for medical imaging analysis, Front. Oncol. 11 (2021) 638182.
[8] J.J. Ma, U. Nakarmi, C.Y.S. Kin, C.M. Sandino, J.Y. Cheng, A.B. Syed, P. Wei, J.M. Pauly, S.S. Vasanawala, Diagnostic image quality assessment and classification in medical imaging: Opportunities and challenges, in: 2020 IEEE 17th International Symposium on Biomedical Imaging, ISBI, IEEE, 2020, pp. 337–340.
[9] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A.C. Berg, W.-Y. Lo, et al., Segment anything, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 4015–4026.
[10] J. Ma, Y. He, F. Li, L. Han, C. You, B. Wang, Segment anything in medical images, Nat. Commun. 15 (1) (2024) 654.
[11] J. Li, Y. Gao, Y. Yang, Y. Bai, X. Zhou, Y. Li, H. Sun, Y. Liu, X. Si, Y. Ye, et al., Fundamental capabilities and applications of large language models: A survey, ACM Comput. Surv. (2025).
[12] A. Matarazzo, R. Torlone, A survey on large language models with some insights on their capabilities and limitations, 2025, arXiv preprint arXiv:2501.04040.
[13] A. Ghosh, A. Acharya, S. Saha, V. Jain, A. Chadha, Exploring the frontier of vision-language models: A survey of current methodologies and future directions, 2024, arXiv preprint arXiv:2404.07214.
[14] E. Nguyen, H. Liu, D. Ruan, Necessity and impact of specialization of large foundation model for medical segmentation tasks, Med. Phys. 52 (1) (2025) 321–328.
[15] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
[16] G. Huang, Z. Liu, L. Van Der Maaten, K.Q. Weinberger, Densely connected convolutional networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 4700–4708.
[17] M. Tan, Q. Le, Efficientnet: Rethinking model scaling for convolutional neural networks, in: International Conference on Machine Learning, PMLR, 2019, pp. 6105–6114.
[18] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., An image is worth 16 × 16 words: Transformers for image recognition at scale, 2020, arXiv preprint arXiv:2010.11929.
[19] M.M.A. Monshi, J. Poon, V. Chung, F.M. Monshi, CovidXrayNet: Optimizing data augmentation and CNN hyperparameters for improved COVID-19 detection from CXR, Comput. Biol. Med. 133 (2021) 104375.
[20] P. Muthukannan, et al., Optimized convolution neural network based multiple eye disease detection, Comput. Biol. Med. 146 (2022) 105648.

[21] P. Rajpurkar, J. Irvin, K. Zhu, B. Yang, H. Mehta, T. Duan, D. Ding, A. Bagul, C. Langlotz, K. Shpanskaya, et al., Chexnet: Radiologist-level pneumonia detection on chest X-rays with deep learning, 2017, arXiv preprint arXiv:1711.05225.

[22] J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2015, pp. 3431–3440.

[23] V. Badrinarayanan, A. Kendall, R. Cipolla, Segnet: A deep convolutional encoder-decoder architecture for image segmentation, IEEE Trans. Pattern Anal. Mach. Intell. 39 (12) (2017) 2481–2495.

[24] O. Ronneberger, P. Fischer, T. Brox, U-net: Convolutional networks for biomedical image segmentation, in: Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5-9, 2015, Proceedings, Part III 18, Springer, 2015, pp. 234–241.

[25] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, H. Adam, Encoder-decoder with atrous separable convolution for semantic image segmentation, in: Proceedings of the European Conference on Computer Vision, ECCV, 2018, pp. 801–818.

[26] J. Chen, Y. Lu, Q. Yu, X. Luo, E. Adeli, Y. Wang, L. Lu, A.L. Yuille, Y. Zhou, Transunet: Transformers make strong encoders for medical image segmentation, 2021, arXiv preprint arXiv:2102.04306.

[27] H. Cao, Y. Wang, J. Chen, D. Jiang, X. Zhang, Q. Tian, M. Wang, Swin-unet: Unet-like pure transformer for medical image segmentation, in: European Conference on Computer Vision, Springer, 2022, pp. 205–218.

[28] A. Bhandari, J. Koppen, M. Agzarian, Convolutional neural networks for brain tumour segmentation, Insights Into Imaging 11 (1) (2020) 77.

[29] C. L. Srinidhi, P. Aparna, J. Rajan, Recent advancements in retinal vessel segmentation, J. Med. Syst. 41 (2017) 1–22.

[30] K. He, G. Gkioxari, P. Dollár, R. Girshick, Mask r-cnn, in: Proceedings of the IEEE International Conference on Computer Vision, 2017, pp. 2961–2969.

[31] M.K. Grzeszczyk, S. Płotka, A. Sitek, Multi-task swin transformer for motion artifacts classification and cardiac magnetic resonance image segmentation, in: International Workshop on Statistical Atlases and Computational Models of the Heart, Springer, 2022, pp. 409–417.

[32] A. Ter-Sarkisov, Covid-ct-mask-net: Prediction of covid-19 from ct scans using regional features, Appl. Intell. (2022) 1–12.

[33] X. Zou, J. Yang, H. Zhang, F. Li, L. Li, J. Wang, L. Wang, J. Gao, Y.J. Lee, Segment everything everywhere all at once, Adv. Neural Inf. Process. Syst. 36 (2023) 19769–19782.

[34] X. Lai, Z. Tian, Y. Chen, Y. Li, Y. Yuan, S. Liu, J. Jia, Lisa: Reasoning segmentation via large language model, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 9579–9589.

[35] H. Rasheed, M. Maaz, S. Shaji, A. Shaker, S. Khan, H. Cholakkal, R.M. Anwer, E. Xing, M.-H. Yang, F.S. Khan, Glamm: Pixel grounding large multimodal model, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2024, pp. 13009–13018.

[36] Z. Wang, Z. Wu, D. Agarwal, J. Sun, Medclip: Contrastive learning from unpaired medical images and text, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing. Conference on Empirical Methods in Natural Language Processing, vol. 2022, 2022, p. 3876.

[37] B. Boecking, N. Usuyama, S. Bannur, D.C. Castro, A. Schwaighofer, S. Hyland, M. Wetscherek, T. Naumann, A. Nori, J. Alvarez-Valle, et al., Making the most of text semantics to improve biomedical vision–language processing, in: European Conference on Computer Vision, Springer, 2022, pp. 1–21.

[38] T. Koleilat, H. Asgariandehkordi, H. Rivaz, Y. Xiao, Medclip-sam: Bridging text and image towards universal medical image segmentation, in: International Conference on Medical Image Computing and Computer-Assisted Intervention, Springer, 2024, pp. 643–653.

[39] C. Li, C. Wong, S. Zhang, N. Usuyama, H. Liu, J. Yang, T. Naumann, H. Poon, J. Gao, Llava-med: Training a large language-and-vision assistant for biomedicine in one day, Adv. Neural Inf. Process. Syst. 36 (2023) 28541–28564.

[40] Y. Luo, J. Zhang, S. Fan, K. Yang, Y. Wu, M. Qiao, Z. Nie, Biomedgpt: Open multimodal generative pre-trained transformer for biomedicine, 2023, arXiv preprint arXiv:2308.09442.

[41] J. Wu, M. Zhong, S. Xing, Z. Lai, Z. Liu, Z. Chen, W. Wang, X. Zhu, L. Lu, T. Lu, et al., Visionllm v2: An end-to-end generalist multimodal large language model for hundreds of vision-language tasks, Adv. Neural Inf. Process. Syst. 37 (2024) 69925–69975.

[42] Y.-C. Chen, W.-H. Li, C. Sun, Y.-C.F. Wang, C.-S. Chen, SAM4MLLM: Enhance multi-modal large language model for referring expression segmentation, in: European Conference on Computer Vision, Springer, 2024, pp. 323–340.

[43] S. Dai, K. Ye, G. Liu, H. Tang, L. Zhan, Zeus: Zero-shot LLM instruction for union segmentation in multimodal medical imaging, 2025, arXiv preprint arXiv:2504.07336.

[44] C. Wei, Y. Zhong, H. Tan, Y. Zeng, Y. Liu, Z. Zhao, Y. Yang, InstructSeg: Unifying instructed visual segmentation with multi-modal large language models, 2024, arXiv preprint arXiv:2412.14006.

[45] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 7132–7141.

[46] Japanese Society of Radiological Technology (JSRT), The Standard Digital Image Database of chest radiographs with and without a lung nodule was developed as part of the research activities of the Academic Committee of the Japanese Society of Radiological Technology (JSRT) in 1998 (*1), supported by the Japan Radiological Society (JRS) and with the cooperation of healthcare facilities in Japan and the United States for providing case materials, 1998, http://db.jsrt.or.jp/eng.php. (Accessed on: 15 June 2024).

[47] Montgomery County, National library of medicine. Montgomery-county-CXR-set, 2021, https://data.lhncbc.nlm.nih.gov/public/Tuberculosis-Chest-X-ray-Datasets/Montgomery-County-CXR-Set/MontgomerySet/index.html. (Accessed on: 10 July 2024).

[48] Shenzhen Hospital, Shenzhen Hospital X-ray Set collected by Shenzhen No.3 Hospital in Shenzhen, Guangdong providence, China. The X-rays were acquired as part of the routine care at Shenzhen Hospital, 2021, https://data.lhncbc.nlm.nih.gov/public/Tuberculosis-Chest-X-ray-Datasets/Shenzhen-Hospital-CXR-Set/index.html. (Accessed on: 15 July 2024).

[49] T. Rahman, A. Khandakar, M.A. Kadir, K.R. Islam, K.F. Islam, R. Mazhar, T. Hamid, M.T. Islam, S. Kashem, Z.B. Mahbub, et al., Reliable tuberculosis detection using chest X-ray with deep learning, segmentation and visualization, Ieee Access 8 (2020) 191586–191601.

[50] M.E. Chowdhury, T. Rahman, A. Khandakar, R. Mazhar, M.A. Kadir, Z.B. Mahbub, K.R. Islam, M.S. Khan, A. Iqbal, N. Al Emadi, et al., Can AI help in screening viral and COVID-19 pneumonia? Ieee Access 8 (2020) 132665–132676.

[51] J. Staal, M.D. Abràmoff, M. Niemeijer, M.A. Viergever, B. Van Ginneken, Ridge-based vessel segmentation in color images of the retina, IEEE Trans. Med. Imaging 23 (4) (2004) 501–509.

[52] A. Hoover, V. Kouznetsova, M. Goldbaum, Locating blood vessels in retinal images by piecewise threshold probing of a matched filter response, IEEE Trans. Med. Imaging 19 (3) (2000) 203–210.

[53] M.M. Fraz, P. Remagnino, A. Hoppe, B. Uyyanonvara, A.R. Rudnicka, C.G. Owen, S.A. Barman, An ensemble classification-based approach applied to retinal blood vessel segmentation, IEEE Trans. Biomed. Eng. 59 (9) (2012) 2538–2548.

[54] D. Jha, P.H. Smedsrud, M.A. Riegler, P. Halvorsen, T. De Lange, D. Johansen, H.D. Johansen, Kvasir-seg: A segmented polyp dataset, in: International Conference on Multimedia Modeling, Springer, 2019, pp. 451–462.

[55] J. Bernal, F.J. Sánchez, G. Fernández-Esparrach, D. Gil, C. Rodríguez, F. Vilariño, WM-DOVA maps for accurate polyp highlighting in colonoscopy: Validation vs. saliency maps from physicians, Comput. Med. Imaging Graph. 43 (2015) 99–111.

[56] R. Kiefer, M. Abid, J. Steen, M.R. Ardali, E. Amjadian, A catalog of public glaucoma datasets for machine learning applications: A detailed description and analysis of public glaucoma datasets available to machine learning engineers tackling glaucoma-related problems using retinal fundus images and OCT images, in: Proceedings of the 2023 7th International Conference on Information System and Data Mining, 2023, pp. 24–31.

[57] J. Sivaswamy, S. Krishnadas, A. Chakravarty, G. Joshi, A.S. Tabish, et al., A comprehensive retinal image dataset for the assessment of glaucoma from the optic nerve head analysis, JSM Biomed. Imaging Data Pap. 2 (1) (2015) 1004.

[58] V.K. Velpula, A. Roy, L.D. Sharma, D. Sharma, Fundus image datasets: A valuable resource for deep neural network-based glaucoma research, Intell. Comput. Anal. Sustain. Energy Environ. (2024) 84–89.

[59] Z. Zhang, F.S. Yin, J. Liu, W.K. Wong, N.M. Tan, B.H. Lee, J. Cheng, T.Y. Wong, Origa-light: An online retinal fundus image database for glaucoma analysis and research, in: 2010 Annual International Conference of the IEEE Engineering in Medicine and Biology, IEEE, 2010, pp. 3065–3068.

[60] O. Kovalyk, J. Morales-Sánchez, R. Verdú-Monedero, I. Sellés-Navarro, A. Palazón-Cabanes, J.-L. Sancho-Gómez, PAPILA: Dataset with fundus images and clinical data of both eyes of the same patient for glaucoma assessment, Sci. Data 9 (1) (2022) 291.

[61] J.I. Orlando, H. Fu, J.B. Breda, K. Van Keer, D.R. Bathula, A. Diaz-Pinto, R. Fang, P.-A. Heng, J. Kim, J. Lee, et al., Refuge challenge: A unified framework for evaluating automated methods for glaucoma assessment from fundus photographs, Med. Image Anal. 59 (2020) 101570.

[62] F. Fumero, S. Alayón, J.L. Sanchez, J. Sigut, M. Gonzalez-Hernandez, RIM-ONE: An open retinal image database for optic nerve evaluation, in: 2011 24th International Symposium on Computer-Based Medical Systems, CBMS, IEEE, 2011, pp. 1–6.

[63] S. Rajaraman, S.K. Antani, M. Poostchi, K. Silamut, M.A. Hossain, R.J. Maude, S. Jaeger, G.R. Thoma, Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images, PeerJ 6 (2018) e4568.

[64] R. Gupta, S. Gehlot, A. Gupta, C-NMC: B-lineage acute lymphoblastic leukaemia: A blood cancer dataset, Med. Eng. Phys. 103 (2022) 103793.

[65] V.T.Q. Huy, C.-M. Lin, An improved densenet deep neural network model for tuberculosis detection using chest X-ray images, IEEE Access 11 (2023) 42839–42849.

[66] S. Rajaraman, G. Zamzmi, L. Folio, P. Alderson, S. Antani, Chest X-ray bone suppression for improving classification of tuberculosis-consistent findings, Diagnostics 11 (5) (2021) 840.

[67] S. Boit, R. Patil, An efficient deep learning approach for malaria parasite detection in microscopic images, Diagnostics 14 (23) (2024) 2738.

[68] G. Madhu, A.W. Mohamed, S. Kautish, M.A. Shah, I. Ali, Intelligent diagnostic model for malaria parasite detection and classification using imperative inception-based capsule neural networks, Sci. Rep. 13 (1) (2023) 13377.
[69] M. Jawahar, L.J. Anbarasi, S. Narayanan, A.H. Gandomi, An attention-based deep learning for acute lymphoblastic leukemia classification, Sci. Rep. 14 (1) (2024) 17447.
[70] N. Sampathila, K. Chadaga, N. Goswami, R.P. Chadaga, M. Pandya, S. Prabhu, M.G. Bairy, S.S. Katta, D. Bhat, S.P. Upadya, Customized deep learning classifier for detection of acute lymphoblastic leukemia using blood smear images, in: Healthcare, vol. 10, no. 10, MDPI, 2022, p. 1812.
[71] A. Shoukat, S. Akbar, S.A. Hassan, S. Iqbal, A. Mehmood, Q.M. Ilyas, Automatic diagnosis of glaucoma from retinal images using deep learning approach, Diagnostics 13 (10) (2023) 1738.
[72] A. Neto, J. Camara, A. Cunha, Evaluations of deep learning approaches for glaucoma screening using retinal images from mobile device, Sensors 22 (4) (2022) 1449.
[73] C.-T. Yen, C.-Y. Tsao, Lightweight convolutional neural network for chest X-ray images classification, Sci. Rep. 14 (1) (2024) 29759.
[74] S. Chauhan, D.R. Edla, V. Boddu, M.J. Rao, R. Cheruku, S.R. Nayak, S. Martha, K. Lavanya, T.D. Nigat, Detection of COVID-19 using edge devices by a light-weight convolutional neural network from chest X-ray images, BMC Med. Imaging 24 (1) (2024) 1.
[75] W. Liu, J. Luo, Y. Yang, et al., Automatic lung segmentation in chest X-ray images using improved U-net, Sci. Rep. 12 (1) (2022) 8649, http://dx.doi.org/10.1038/s41598-022-12743-y.
[76] H. Yuan, C. Hong, N.T.A. Tran, X. Xu, N. Liu, Leveraging anatomical constraints with uncertainty for pneumothorax segmentation, Health Care Sci. 3 (6) (2024) 456–474, http://dx.doi.org/10.1002/hcs2.119.
[77] X. Liu, H. Tan, W. Wang, Z. Chen, Deep learning based retinal vessel segmentation and hypertensive retinopathy quantification using heterogeneous features cross-attention neural network, Front. Med. 11 (2024) 1377479, http://dx.doi.org/10.3389/fmed.2024.1377479.
[78] R. Yang, S. Zhang, Enhancing retinal vascular structure segmentation in images with a novel design two-path interactive fusion module model, 2024, arXiv preprint arXiv:2403.01362.
[79] H. Xue, Y. Liu, M. Li, et al., A lighter hybrid feature fusion framework for polyp segmentation, Sci. Rep. 14 (2024) 23179, http://dx.doi.org/10.1038/s41598-024-72763-8.
[80] J. Li, Q. Xu, X. He, Z. Liu, D. Zhang, R. Wang, R. Qu, G. Qiu, Cfformer: Cross CNN-transformer channel attention and spatial feature fusion for improved segmentation of low quality medical images, 2025, arXiv preprint arXiv:2501.03629.
[81] X. Guo, J. Li, Q. Lin, Z. Tu, X. Hu, S. Che, Joint optic disc and cup segmentation using feature fusion and attention, Comput. Biol. Med. (ISSN: 0010-4825) 150 (2022) 106094, http://dx.doi.org/10.1016/j.compbiomed.2022.106094, URL: https://www.sciencedirect.com/science/article/pii/S0010482522008022.
[82] Y. Yi, Y. Jiang, B. Zhou, N. Zhang, J. Dai, X. Huang, Q. Zeng, W. Zhou, C2FTFNet: Coarse-to-fine transformer network for joint optic disc and cup segmentation, Comput. Biol. Med. 164 (2023) 107215.
[83] R. Bhattacharya, R. Hussain, A. Chatterjee, D. Paul, S. Chatterjee, D. Dey, PY-net: rethinking segmentation frameworks with dense pyramidal operations for optic disc and cup segmentation from retinal fundus images, Biomed. Signal Process. Control. 85 (2023) 104895.
[84] M.T. Mahmood, O.N. Ucan, Data and image processing for intelligent glaucoma detection and optic disc segmentation using deep convolutional neural network architecture, Discov. Comput. 28 (2025) 73, http://dx.doi.org/10.1007/s10791-025-09587-1.